

```
import matplotlib as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Start coding or [generate](#) with AI.

```
import seaborn as sns
import pandas as pd

# Load the 'tips' dataset from seaborn
df = sns.load_dataset('tips')
```

```
df.head()
```

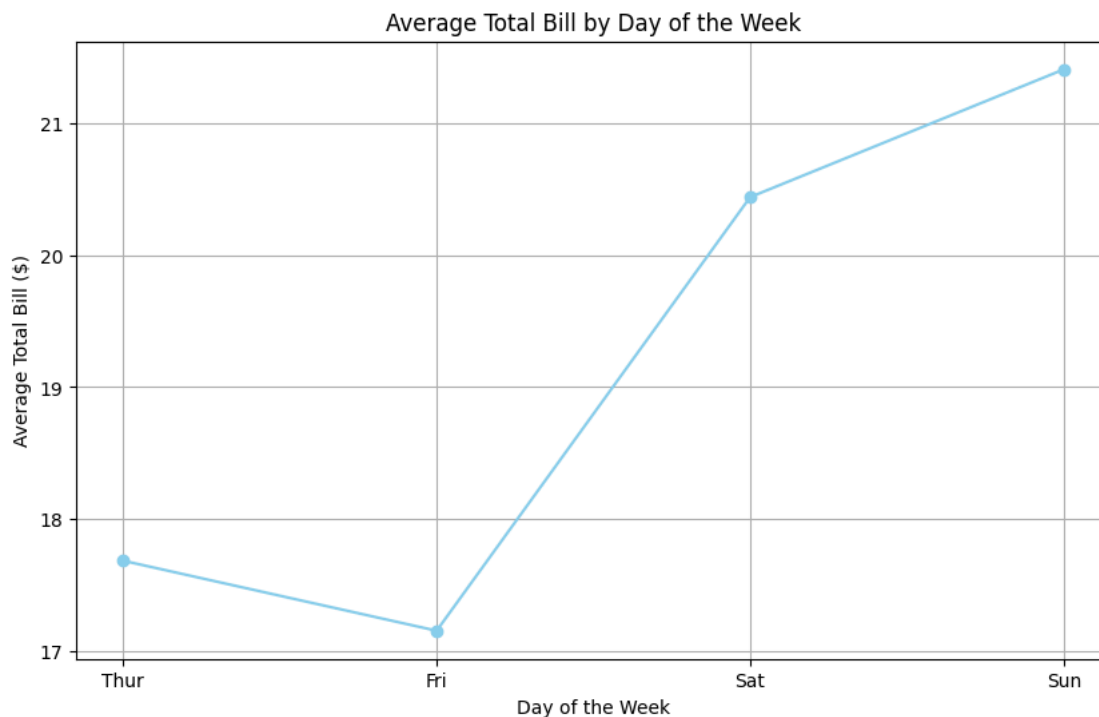
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
import matplotlib.pyplot as plt
```

```
daily_average_bill = df.groupby('day', observed=False)['total_bill'].mean().reindex(['Thur', 'Fri', 'Sat', 'Sun'])
```

```
plt.figure(figsize=(10, 6))
plt.plot(daily_average_bill.index, daily_average_bill.values, marker='o', linestyle='--', color='skyblue')
```

```
plt.title('Average Total Bill by Day of the Week')
plt.xlabel('Day of the Week')
plt.ylabel('Average Total Bill ($)')
plt.grid(True)
plt.show()
```

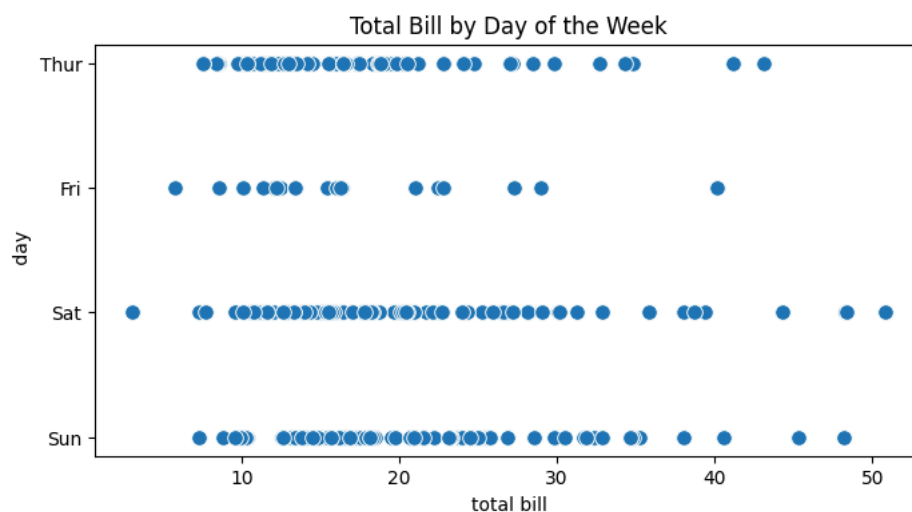


```
# to plot scatter plot

plt.figure(figsize=(8, 4))

sns.scatterplot(x='total_bill', y='day', data=df, s=70)

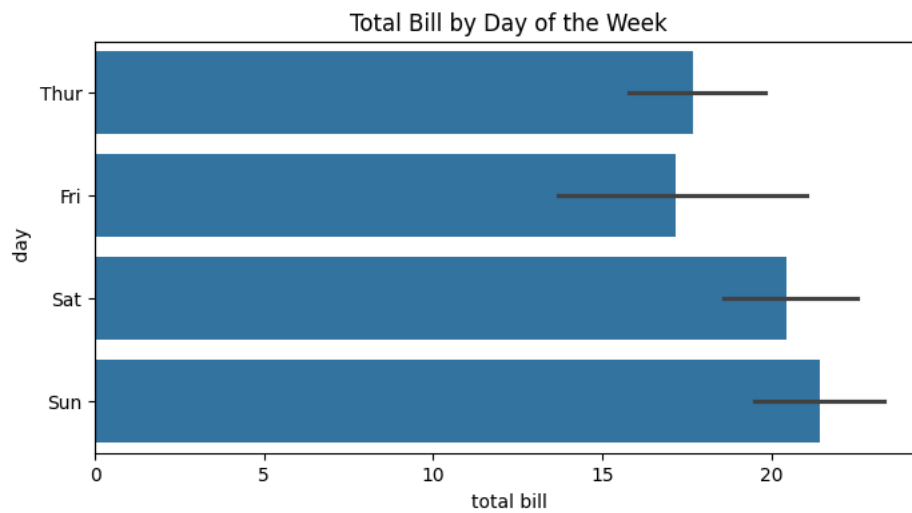
plt.title('Total Bill by Day of the Week')
plt.xlabel('total bill')
plt.ylabel(' day')
plt.show()
```



```
plt.figure(figsize=(8, 4))

sns.barplot(x='total_bill', y='day', data=df)

plt.title('Total Bill by Day of the Week')
plt.xlabel('total bill')
plt.ylabel(' day')
plt.show()
```

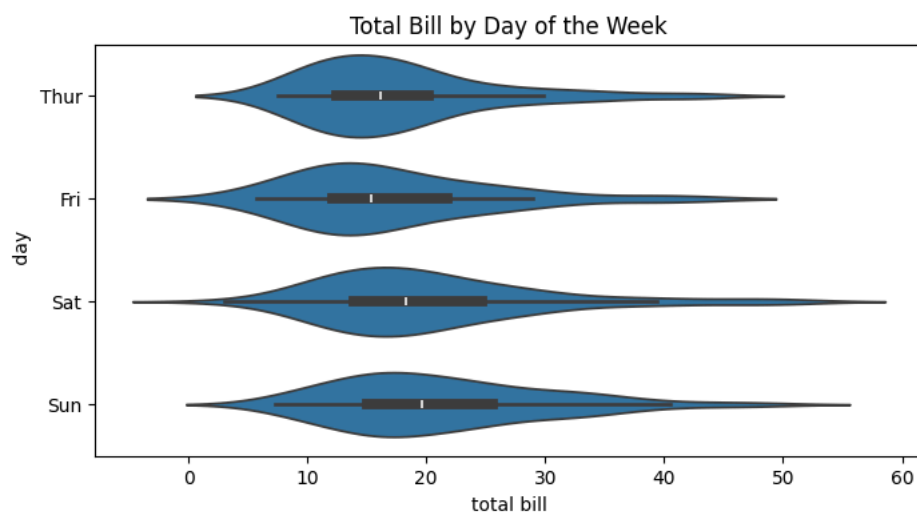


```
plt.figure(figsize=(8, 4))

sns.violinplot(x='total_bill', y='day', data=df)

plt.title('Total Bill by Day of the Week')
plt.xlabel('total bill')
plt.ylabel(' day')
plt.show()
```

This violinplot visualizes the distribution of 'total_bill' for each 'day' of the week.
 # It shows the probability density of the data at different values, smoothed by a kernel density estimator.
 # Unlike a boxplot, which only shows summary statistics, a violinplot shows the full distribution of the data.
 # The wider sections represent a higher probability density.

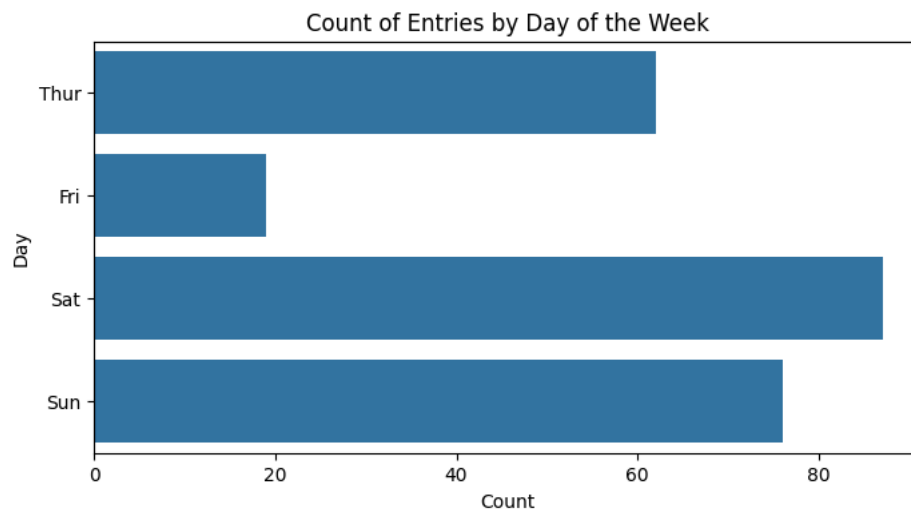


```
plt.figure(figsize=(8, 4))

sns.countplot(y='day', data=df)

plt.title('Count of Entries by Day of the Week')
plt.xlabel('Count')
plt.ylabel('Day')
plt.show()
```

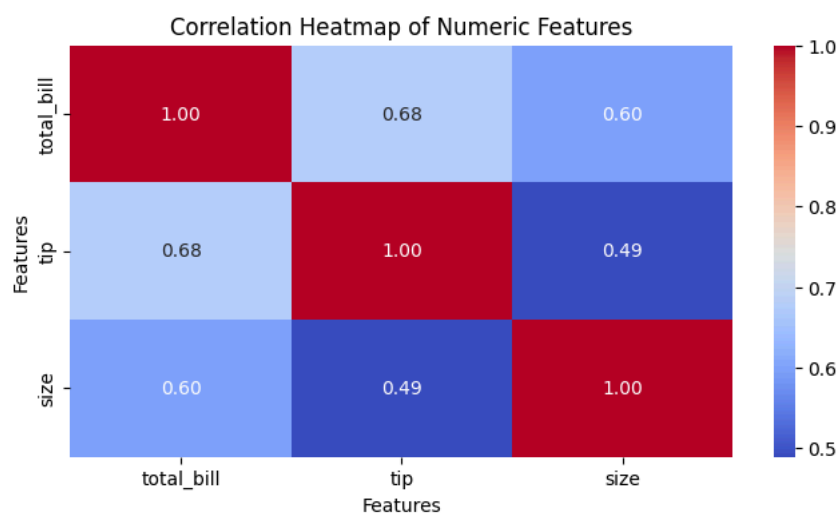
This countplot displays the number of observations (or entries) for each category on the 'day' axis.
 # It's used to visualize the frequency distribution of a categorical variable.



```
plt.figure(figsize=(8, 4))
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap='coolwarm', fmt=".2f")
```

```
plt.title('Correlation Heatmap of Numeric Features')
plt.xlabel('Features')
plt.ylabel('Features')
plt.show()
```

The correlation heatmap visualizes the linear relationships between the numeric features in the dataset.
 # 'annot=True' displays the correlation values on the map.
 # 'cmap='coolwarm'' sets the color scheme, with 'cool' colors typically indicating negative correlations and 'warm' colors
 # 'fmt=".2f"' formats the annotation values to two decimal places.

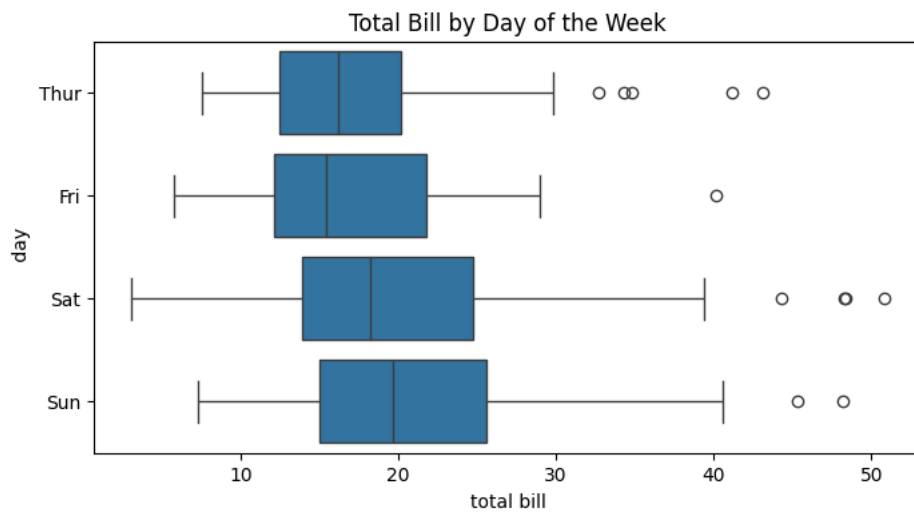


Double-click (or enter) to edit

```
plt.figure(figsize=(8, 4))
sns.boxplot(x='total_bill', y='day', data=df)
```

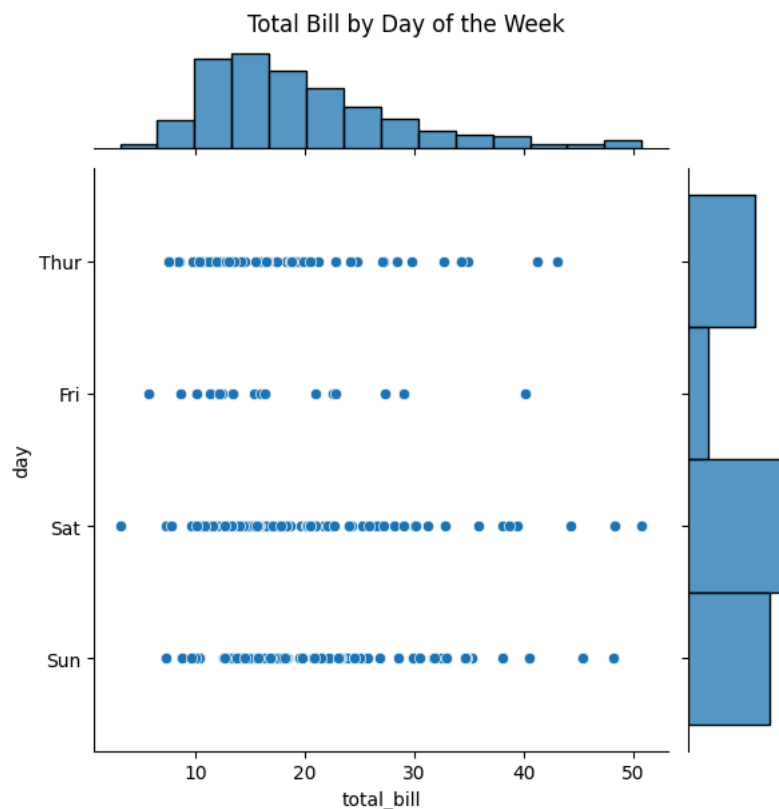
```
plt.title('Total Bill by Day of the Week')
plt.xlabel('total bill')
plt.ylabel(' day')
plt.show()
```

This boxplot visualizes the distribution of 'total_bill' for each 'day' of the week.
 # Each box represents the interquartile range (IQR), with the line inside indicating the median.
 # The 'whiskers' extend to 1.5 times the IQR from the box edges, showing the spread of data.
 # Points beyond the whiskers are considered outliers.



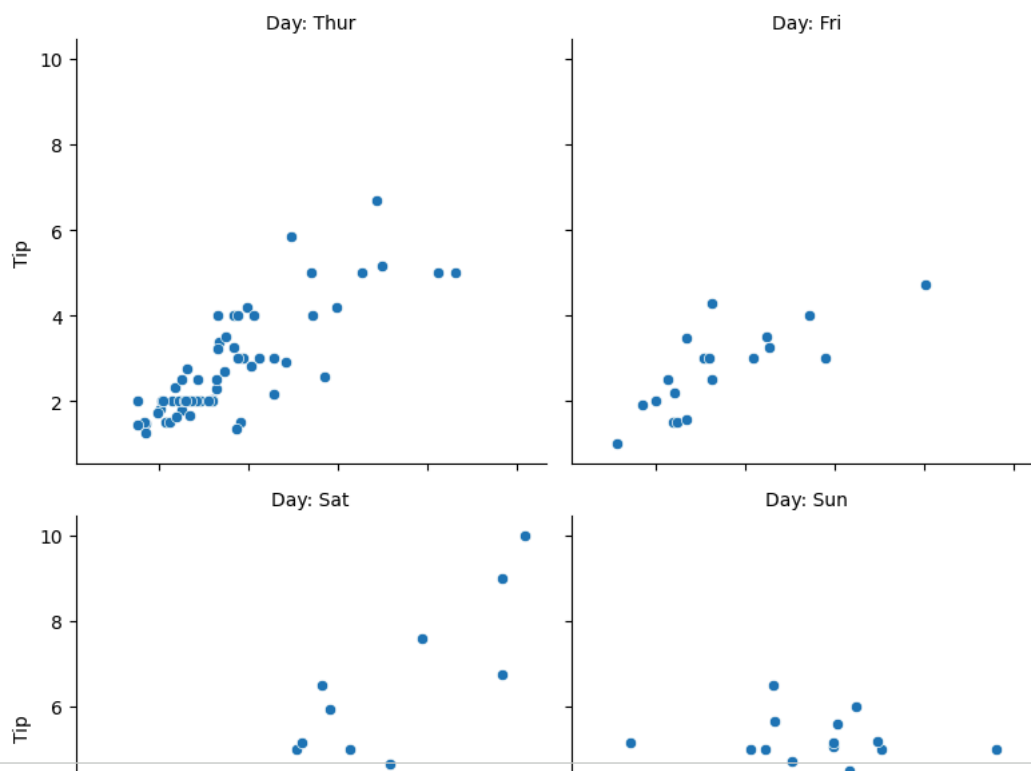
```
g = sns.jointplot(x='total_bill', y='day', data=df)
g.fig.suptitle('Total Bill by Day of the Week', y=1.02)
plt.show()
```

A jointplot displays the relationship between two variables and their individual distributions.
 # The central part shows a scatter plot of 'total_bill' vs 'day'.
 # The marginal axes show histograms for each variable independently.



```
g = sns.FacetGrid(data=df, col='day', col_wrap=2, height=4)
g.map(sns.scatterplot, 'total_bill', 'tip')
g.set_axis_labels('Total Bill', 'Tip')
g.set_titles(col_template='Day: {col_name}')
plt.suptitle('Total Bill vs Tip by Day of the Week', y=1.02)
plt.tight_layout()
plt.show()
```

Total Bill vs Tip by Day of the Week



```
sns.pairplot(data=df, hue='day')
plt.show()
```

A pairplot creates a grid of pairwise relationships between variables in a dataset.
 # The diagonal plots show the distribution of each variable.
 # The off-diagonal plots show scatter plots of each pair of variables.
 # 'hue='day'' differentiates the data points by the 'day' column using different colors, allowing for visual comparison across days.

